

Hi, everyone. Today we're going to talk about two topics, normalization and regularization.

In the past few lectures, we've talked about how to build the internals of a neural network library.

Now we're going to talk a little bit about some of the tricks and architectures you can use to help you train these things more efficiently and build better and easier to train architectures.

Normalization and regularization are two somewhat different aspects to modern deep networks.

We're going to talk about both of these because they're somewhat related, and as we'll see later in the last section, these things all do have some degree of interaction, as well as also interact with things like initialization and optimization.

To start off with, we're going to talk about normalization, then we're going to talk about regularization, and then we're going to talk about the interaction of all these things, including optimization, initialization, et cetera.

All right, so let's jump right in and start with normalization.

If you remember from a few lectures ago, the initial weights that we chose for a deep network were very important.

So for example, you may have remembered that we initialized our weights with Gaussian random variables.

So often, it was we initialized our weights to be random.

Maybe sometimes uniform or Gaussian random variables.

But the variance of these Gaussian random variables mattered a lot.

So this notation here just says that we're initializing our weights randomly.

We're initializing them with random samples from a Gaussian distribution with mean 0 and variance of $\frac{c}{n}$ over n , where n is the number of the dimensionality of the input to this layer.

And remember that for a ReLU network, well, first of all, remember two things.

the choice here of c is very important.

And it turns out that for a ReLU network, the choice of c equals 2 is sort of the right choice, because it maintains the variance of the activations as you go deeper and deeper in a network.

We'll actually see that in this figure here.

When you choose the variance here to be $\frac{2}{n}$, then this corresponds to this line here, and it maintains variance of the layers.

I'll discuss what this figure is showing in a second, though.

But what I also want to show here is that we can also initialize with other variances, too.

So we can initialize the variance of, say, $\frac{3}{n}$ or $\frac{1}{n}$, et cetera.

And these are actually pretty close.

These are within the same factor of the same rough order of magnitude as $\frac{2}{n}$.

But they lead to very different behavior.

And so to show that, I'm actually I'm going to show these plots below here.

So what these plots show here is I'm initializing a 50-layer deep network and applying it to, in this case, MNIST.

Doesn't really matter what I'm applying it to here, but it will matter when I'm training these networks a little bit.

So what I'm showing in these plots here is the layer of the network.

So this x-axis here shows the layer of the network.

This network is 50 layers deep.

And on the y-axis, at least in this plot, I'm showing, the first plot, I'm showing the activation norm.

So what's the norm of the z_i term?

So this term here basically would be like the norm of the different z_i terms as you go through the network.

And what I'm showing here on this next plot is the gradient norm.

So that would be the norm of the gradient values as you go through the network.

So something like the norm of the gradient with respect to w_i of our loss.

So abbreviate it like that.

So these are the two things that I'm showing here.

These are both actually sort of L2 norms in this case.

All right, so what we see here is sort of two interesting things.

The first is that in the case, as we kind of expected actually, in the case where we pick the right activations

for the ReLU network, our norms stay roughly the same throughout optimization, or, sorry, throughout the layers of the network, I mean.

They, you know, the norm hovers around, you know, the activations themselves kind of hover around kind of a similar value throughout the whole layer, the whole depth of the network.

But if we pick other values, if I pick one over n , then this is, in some sense, too small a weight.

And so what this means is that the activations decrease in norm a lot throughout the depth of the network.

And similarly, if I pick a larger initialization, the values increase over the depth of the network.

That's sort of the first thing to notice.

And the second thing to notice is that the gradients, those actually are roughly static because just the, you can kind of look at this if you want,

but the formula for the gradient, remember, You sort of use both the backward pass and the forward pass to compute the gradient with respect to the weights.

And terms cancel, such that these things actually remain constant throughout the whole network.

But the big difference, of course, here, is that if you initialize with sort of the right, or two over n , call that the right or the wrong,

depending on your perspective, but with the right, sort of the correct initialization to maintain the norm of the activations across depth,

you get kind of a reasonable scaling to your gradients, whereas for larger values, you get sort of much larger values of your gradients here.

So gradients of 10^4 for the three over n case or 10^{-8} for the one over n case.

And what this corresponds to, and this is sort of the punchline here, is that you may think that sort of the initialization Initialization, it matters, but it doesn't matter that much because, after all, you actually are optimizing these things.

So, you know, maybe after a few iterations, these weights will be kind of fixed, and it doesn't really matter how you initialize them.

They'll all end up the same.

But this is definitely not true in deep networks.

So, for example, if I pick 3 over n as my variance to initialize with, I have a very big norm for my gradients here.

and what ends up happening is that my loss just goes to not a number I think if you've ever tried with

I've tried to optimize deep networks I'm sure you've in some sense or in some cases had the experience of training a network and all of a sudden the loss becomes

NaN or your weights become NaN values basically you have numerical overflow sometimes everything blows up and things just go wrong and what's happening here is that the norm of the gradients are just too large and so as I optimize these things my weights just sort of start to go unstable, and things diverge, and you get NaNs.

And no one likes this.

On the flip side, if you try to optimize starting at the $\frac{1}{n}$ point, nothing ever happens. You make no progress.

So you just sort of stall out for a while.

And whereas, of course, if you use $\frac{2}{n}$, in this case it does work, and you are able to optimize even this 50-layer deep network.

It's not actually, I should emphasize, you don't get a huge benefit from training a 50-layer network on MNIST, but it doesn't fast work. It's only on a fully connected 50-layer network, but it does

it doesn't fast work. And so this, the big intuition I want to emphasize here is that I for a long time had this notion that the incorrect initialization would be quickly in

in some sense, fixed by optimization, right?

Because who cares how we initialize things, too?

In convex optimization, for example, it doesn't really matter how you initialize things because your weights kind of go to the correct point

no matter what, as long as you're not sort of too far out of bounds.

But deep learning is very different.

And when you're optimizing deep networks, we often find that the initial values for our weights matter a whole lot because otherwise things will just never train.

It doesn't matter how long you train and how many gradient steps you take or how many SGD iterations you take.

They just never train or they diverge.

And I want to emphasize this problem actually goes much deeper than just this example of sort of these failure cases of either not making progress at all or of blowing up and becoming NaN values.

Because actually, it turns out that if I even initialize things to a more narrow scale, we actually get similar effects here.

So in this case, I'm showing three more examples with the initialization chosen, such that you still have sort of a,

this is initialization of $\frac{1.7}{n}$ for the variance, which is a little bit less than two, then you have two as before, and then you have $\frac{2.3}{n}$.

And these networks actually all will train.

They will train to some degree of loss.

You can get the loss down from this.

But the point I want to make here is that the behavior, or the effects you have at initialization, they persist throughout the whole network.

So I'm gonna train this network.

I'm gonna train all three of these networks, actually, to 5% error on MNIST.

And what you see here is that there's a lot of interesting effects to this plot, actually.

One thing you'll see is that the activation norms actually do change from the initial version.

So you have activations that, no matter what your initialization is, you have activations that actually in the later stages of the network are a bit more.

They're a bit higher here than they are in the initial layers.

And there is some difference in the different ones.

So the activations of the blue initialization do remain a bit lower than the other ones.

But those actually do kind of in some sense become at least fairly similar.

The gradients also change by the layers, and some effects persist from the initialization, but they also kind of stay similar.

The point I actually want to highlight here is an interesting point, and I've always found this very, very surprising.

It's not really having an effect here on error, but the point that I want to emphasize here is that if you look at the actual weights after training, you cannot differentiate between these two figures.

I actually had to check this several times to make sure I wasn't making a mistake here, because I've always found this very surprising.

is that when you train a deep network, it turns out that your weights change very little compared to their initial values, or rather, the way I'll put it, obviously, the weights are changing.

I mean, these values are different.

I'm just plotting the norm of the weights here by layers.

But this line here is actually essentially indistinguishable from this one.

And the point is not that the weights don't change.

Weights, of course, do change.

But in some sense, they change in a relatively small degree relative to their initialization, at least for these very deep networks, trained in this case with SGD.

Now, this doesn't always happen.

There are some cases where you get potentially more.

And if you look at certain dimensions, and this is just plotting their norms, so the direction of the weights can still change while sort of keeping a similar norm.

But this is a really interesting effect of deep networks is that it challenges our normal ideas of optimization and what the weights do in optimization.

And so these things have to be taken into account when you think about these networks.

Okay, but moving on from this, the main point I want to emphasize here is that initialization matters,

and initialization matters insofar as it affects the relative norms of the activations and their gradients over time, right?

That's sort of the high-level takeaway I want to make from these three sets of slides here.

So this one here, and then this figure here.

Two sets of slides here.

Okay, so what can we do about this?

Well, one obvious idea is that initialization does matter, and it matters amongst other reasons because when you initialize differently, the norms of your activations may no longer be the same across training.

But the high-level thing to remember here, and now as you probably have experienced a lot in the last couple lectures, a layer in deep networks can really do anything at all.

So we can have a layer that of course computes a linear function, a matrix times the activations.

We can have a layer that computes a nonlinear activation.

But we can do whatever we want in layers.

Layers are just differentiable functions that we plug in and use in our deep network library.

And so the key idea is that these problems of the weights of activations varying over the course of the depth of a network,

We could add a layer, in some sense, that just kind of fixes this.

There's nothing preventing us from building a layer.

If we believe that the norm of the layers kind of changing over time due to bad initialization is a problem, let's just go ahead and add a layer that kind of fixes this.

And this is the key idea of normalization layers.

So what we're going to do in the first instance of a normalization This is a technique called layer normalization, or just layer norm, is kind of to be the obvious thing.

Let's just take our activations at each layer and let's just normalize them to have some standardized mean and some standardized variance.

It's a very natural idea because this will then prevent them from having this bad effects kind of as we go through the later layers of a network.

So the way I'm going to write this is I'm going to write this as we have sort of our initial potential activation here.

I'm going to write $\hat{z}$ here as sort of just for convention, I'm going to write $\hat{z}$ here as kind of our tentative layer before normalization.

So that's going to be just equal to our nonlinearity in this case times a linear function of our previous layer plus a bias term.

This is just our simple fully connected layer here.

We could, of course, also have some more structure in this layer, but we won't worry about that.

Similar things apply, of course, no matter what the structure of the network is.

The simplest thing we can do in some case is just take this layer and ensure that it has some standardized mean and variance, and so let's just choose to, you know, in our argument before, we wanted to choose our initialization such that the activations had

mean 0 and variance 1, so rather than worrying too much about initialization, let's just actually enforce our layer to have mean 0 and variance 1.

And the way we're going to do that is we're going to set our next actual layer just be equal to $\hat{z}_i$ plus 1 minus its expectation.

So minus the expectation, sorry, the expected value of $\hat{z}_i$ plus 1, where here the expected value and also the variance.

This is just an empirical expected value.

Remember, this is a vector here.

So this would just be equal to, say, there are n elements in this vector.

This would be equal to $\frac{1}{n}$ times the sum from i .

Well, I shouldn't use i there because I have the other i index that sum from j equals 1 to n of $\hat{c}_i$ plus 1.

And I'm going to use this notation here to say like the j th element of this vector.

Remember, $\hat{z}_i$ plus 1 is itself a vector.

So we have this kind of a bit odd notation of having two subscripts, right?

indicates the layer it is, which I'm doing just for convention.

We usually use these i 's to refer to layers.

So if I wanted to refer to the j th element of that vector, I'm going to use this notation here of putting parentheses around it and then also using a separate subscript there.

So this is just going to be the empirical expected value.

Then my layer norm term would take this whole thing and divide it by the standard deviation of this vector.

So I'm going to write that actually as the square root of the variance of $\hat{z}_i$, where again this is just the empirical variance.

This would just be equal to the variance of $\hat{z}_i$ plus 1, would just be equal to $\frac{1}{n}$ of the sum from j equals 1 to n of $\hat{z}_i$ plus 1 j minus the expected value of $\hat{z}_i$ plus 1.

We're not going to worry if this is a biased or unbiased estimate or things like that, it's the value of this thing squared.

Okay, so then to normalize a vector, you normalize it by the square root of its variance to make it equal to one, and so we're just going to do that.

Now one last thing that I'll mention is that there is a chance that, say, if you happen to be in this unlucky situation where you've applied a ReLU to your vector, what if all

the values of this term here are zero?

all the values of the inner term just happen for some example, for some example to be negative, and so if these things are all zero, you would actually be dividing by zero.

So a very common thing to do then is just to add a little small epsilon, where epsilon is something like, you know, say it's something like

10^{-5} or something. All that matters is that you make it some small constant such that you don't get numerical underflow, and you divide actually by the variance plus epsilon square root, and you divide by that thing, because that thing will now always be positive.

Okay, and that's the idea. That's it. We've just forced our layers to be equal to essentially have the same norm at each layer, and this does in fact fix the problem we had

in the previous slide. So what I'm doing again here is showing you a similar initialization where you initialize with variance $\frac{1}{2n}$, $\frac{1}{n}$, and $\frac{3}{n}$. And what we see here

is that you get roughly the same weight at each layer of this network. There's some slight variation here. Don't worry about that too much. That's just because you're applying things like

ReLU's also, and also the first layer here is a larger size for MNIST, so it has a bit of a difference there in its activation norm.

But this is essentially what you're doing here.

You are just normalizing each layer.

And that problem of exploding or going to zero kind of activations is solved.

These activations stay all around kind of the same rough value here.

The gradients are also very similar across different initializations.

So we fixed the problem of having a gradient that depends on the initialization a lot.

Kind of oddly enough, there's more variability to the gradients.

If you remember a few slides back, we had very similar gradients all the way through.

Because you're sort of forcing, because you're introducing initial terms into the gradient here, into the backward pass, you actually have to give up that property a little bit.

But the gradients aren't blowing up here.

This is not that large a difference, arguably.

There's not that large a difference, and they're all the same, roughly the same, regardless of the initialization.

So that's just an illustration of layer norm here.

But there is a problem.

And actually, before I go into some of the potential problems, I should say that layer norm is very widely used.

So for example, transformer architectures, which we'll cover in later classes, which are in some sense the dominant architecture in a lot of deep learning these days, they They widely use layer norm as their normalization technique.

And so the simple approach is extremely common in modern deep learning.

And it does, in fact, prevent these big blowups in activations happening.

And in some sense, it's sort of the easy fix.

You have to worry much less about initialization if you do things like this.

Now, at the same time, I should also mention that for standard fully connected networks, It is often harder to train networks to low loss when you add layer norm.

And there's a few reasons for this.

One of the reasons, it's hard to say exactly the reasons.

A lot of these things are, and I'll get to this later in this lecture, are not fully understood.

But one of the reasons here is that layer norm is taking each individual example and forcing its norm of the activations, including, let's say, the last layer, to be equal to 0, or the mean of that activation is equal to 0, and the standard deviation of that activation is equal to 1.

And it might be that, in fact, the relative norms of the different examples, say a digit of a 1 versus a digit of a 0, maybe the relative norms

and their variances are actually a pretty good feature to classify things.

It's very possible that the relative sizes may include some information that we want to actually keep and preserve.

That is just honestly one reason why these networks can be hard to train.

There are many reasons why they might be hard to train.

But what's been found in practice, at least for just a fully connected MLP, it's often harder to train these networks when you introduce layer norm in them than it is without layer norm.

So we're sort of, you know, maybe not quite getting everything that we want out of this.

And so because of this, we're actually going to introduce a different kind of normalization, which is also useful in many scenarios here, which seems like an odd idea.

And so I want to emphasize before I start here that this seems like a very odd thing to do.

But hopefully it will sort of make some amount of sense the way I'm describing it here.

So let's consider...

So before we were considering, when I defined the equation for layer norm, I was considering kind of an independent normalization that I would apply to each layer of a single...

of the network applied to a single example.

So I would take a single input example, look at its activations throughout the network, and I would normalize each of these activations.

But let's consider now the matrix form of our updates.

So remember, we wrote it like this, where we have a whole matrix now, or say, in this case, probably all the examples in a mini-batch.

So Z , a capital Z in its rows contains the examples of each, many examples corresponding to all the examples in that mini-batch.

And so in this setting, so if we sort of define Z in matrix form, like we do anyway when we run these things, then layer norm corresponds to taking a single example, which are the rows of

this matrix, and normalizing the rows, right? But viewed as a matrix, there are many different ways that we could actually kind of get standardization across these features.

And in fact, if you kind of do think about sort of standard normalization of features for a lot of kind of standard machine learning algorithms,

an equally common thing to do is to normalize not just the sort of each row of an entry, but it's also very common to actually normalize the columns of these things too.

So what if instead of normalizing the rows, we normalize the columns instead?

So we actually do something that seems kind of odd, but we normalize kind of think of each sort of column of this vector as sort of a single activation,

kind of like a single feature. What if we just normalized those values across the whole mini batch. This is a technique called batch normalization. And it is also extremely common to use in

practice in deep networks and also provides in many cases a huge benefit for the trainability of these networks. And again, I want to emphasize here that there are a lot of ways of motivating

what we're doing here. But in some level, all we are doing is we are just trying to ensure that the activations of our network don't blow up as we go deeper and deeper into

the network, that we maintain some kind of control over them, and we can just as easily normalize over the rows to do this as normalize the columns over this.

But this form, normalizing over the columns here, maintains the fact that different rows, different examples, could have different norms and different values, sizes for their activations as you go deeper into the network.

that might be a useful discriminative feature.

And so this preserves that while still preserving some degree of normalization of your layers.

And as I said, there's a sort of an obvious analog here to what we often do to prepare data for use in classical machine learning methods.

Oftentimes, we will normalize the features, normalize the columns of our data matrix, and this sort of takes that to the extreme

where we're doing that for each layer in our network, for each mini-batch of our example.

And this is a technique called batch norm.

And so this is very, very common.

People often do this.

And if you've played with these networks at all, you've likely seen networks that use batch norm of some kind.

But I will add, and this fixes the problem, too.

I won't show too many plots here, but it does fix the problem, just like layer norm does.

Of course, by definition, it fixes the problem.

But what I actually want to focus on here is an additional aspect to batch norm that is very common to fix an odd problem in batch norm.

So one thing that is kind of strange, when you normalize, like we did here, by column, you are making the total predictions of your network

in some sense depend on the mini-batch, right?

With layer norm, you're applying your functions just to each example in your batch.

Sorry, you're applying it to each example, period.

You're going to apply this function to one example, the next example in your batch.

It's the same regardless of what examples you pick in your batch.

But the oddity about batch norm is that normalizing your activations by the batch actually introduces a dependency.

And you can just think of this in terms of the computational graph, right?

It introduces a dependency between all the examples in the batch.

In other words, my predictions on this example here, sort of the example number one, whatever I output in that example, will actually, through this mechanism, depend

on my predictions or what the example was in the second row, right?

So this example here depends on this example here.

And that's very strange, right?

You don't want a network where the output of the network depends on other examples besides the one you're actually classifying, right?

That would be very odd.

Maybe it's not that odd, to be honest.

Maybe we actually want to adopt that, but at least, naively speaking, this is a very odd effect.

And so what's commonly done in BatchNorm is to...

when you apply batch norm at test time, you train normalizing by these batches here.

But when you actually apply batch norm at test time, you don't use the actual norm of the batch.

You don't use the batch statistics to normalize your example.

You use computed averages of the mean and variance you've seen over kind of all your examples so far to actually compute that.

And so what happens here is you sort of form these empirical, what we call kind of empirical running means or running averages.

I shouldn't use the word mean because mean implies kind of it's the mean term here.

But running averages of these mean and variance terms.

And so what I mean by that is you'll form sort of one of these μ hat terms.

And here μ hat, for example, could be just something like, as I'm going, my μ hat i plus 1 variable would equal some sort of term β times my old.

I'll just do the notation of setting it equal to as I'm running my network here.

It would be equal to the old estimate of my mean plus something like $1 - \beta$ times my actual expectation of, say, the z hat i plus 1.

So this is sort of like momentum kind of running.

What they actually technically are is exponential moving average estimates, but they are kind of running estimates of what the mean of these parameters looks like.

And similarly for the variance, just averaging the variance over time.

And so what we do then in our update is instead of producing our update just based upon, at test time at least, based upon the actual mean

and variance of our batch, we subtract off this running estimate of the mean and the variance.

So what we do is we say, again, remember our j -th index here.

So this is the j -th column for an example in our batch.

That would be equal to, in this case, $\hat{z}_{i+1,j}$ minus not the empirical expectation of that batch,

but minus this running mean, minus $\hat{\mu}_{i+1}$, the j th column, and then divided by the square root of a similar term.

I won't write what this is, it's very similar, but the empirical estimate of our variance for that feature, J , and then plus an epsilon term again.

And that is the standard update you do at test time.

So to be clear, with batch norm, at training time, the update looks very similar to the layer norm update,

but just what it means is different, right?

So this would be the actual expectation over the batch, this would be the variance over the batch, But then at test time, you run it with these moving averages instead.

And in fact, when you implement your, you will implement in your neural network library the batch norm layer.

And you will have to sort of track these things as you go.

And there are all sorts, by the way, of very weird oddities that come up when you do this.

I've seen all sorts of bugs where people run their network in training mode and sort of, even though they're not training the network, not taking gradient steps.

For example, PyTorch will actually update the running means and variances when you're running in training mode.

Batch Norm causes a lot of problems too, but it also sort of solves a lot of problems.

So there's lots of pluses and minuses to things like BatchNorm, and you'll have to kind of just get used to these things in practice.

But you will implement this in your homework when you implement the neural network library.

All right. Let's move on to the next topic now, which is regularization.

And again, this might seem a bit odd to put together because this is in some sense a different topic, but regularization and normalization are in some sense similar.

They are kind of features that we add to the networks to either make it perform better or to optimize better and things like this.

And they end up having weird interactions that make a lot of sense to talk about these things together.

All right, so to motivate regularization, I'm going to use the fact that typically deep networks, and this is true even for sort of simple two-layer networks

for the cases you've used in the homework so far, they are typically what are called overparameterized models.

And this means the networks have more parameters, more weights, more individual weight values than the number of training examples in the data set.

And this means, and we can even formalize, I won't formalize this, but it can be made formal under certain assumptions, is that they are capable of fitting the training data exactly.

That means that when you optimize one of these networks, they are capable of achieving essentially zero training error, or something at least very, very close to zero training error in practice.

And in the traditional way of thinking about machine learning and statistics, again, with a few big caveats here,

I don't want to make too blanket statements, but to go ahead and still make a blanket statement.

Traditional machine learning or statistics should imply, in some sense, that these models overfit the training set.

What that means is that they will perform well on the training set, but they will not generalize well.

Essentially, when you give it a new test set, they will not perform well on this new test set. This is sort of the classical model of thinking.

Now, of course, we know from a fact that these models sometimes do work well, despite this seeming kind of oddity about being overparameterized and still performing well.

To be clear, there are statistical very formal models for why this can work.

And I'm not trying to imply here that this is some big mystery.

But I also do want to say that there actually is some mystery about this still.

We don't quite understand everything still about this.

But from this classical perspective, it does seem like our models that we have for deep networks, they're just too complex.

And what's likely is that they're going to fit the training data exactly and not fit, not be able to give us good predictions on new examples, which is of course what we really want.

And so to counteract this problem, the traditional way of dealing with this challenge in statistics or in classical machine learning is to apply some form of regularization to our problem.

And speaking very informally here, regularization is in some way the process of limiting the complexity of our function class in order to ensure that our models

generalize better, right? And so this is sort of my informal definition of regularization is about limiting the complexity of

the function class. So our functions are very complex, they have more parameters than they have examples, they can overfit very easily,

and regularization essentially is the process of controlling that complexity so that our networks generalize better.

Now, there are two forms of regularization that are very commonly talked about in deep learning.

And the first of these is what's called implicit regularization.

So this refers to regularization that kind of comes about not through any explicit means, but through kind of just the nature of the architectures or algorithms already considered.

And by the way, this term is vastly overused, I think, in deep learning, sometimes for things that are not really regularization, but you see it everywhere.

But there is a real value in thinking about some of the processes that we use in deep learning as being a form of regularization, even though they are not explicitly trying to control the complexity of our function.

And the best example, I think, is actually stochastic gradient descent itself.

and just the way we optimize these networks.

Maybe more formally, SGD with a given weight initialization.

Because when we say that deep networks are overparameterized, what we really mean is that function class, the class of all deep networks, can easily overfit.

There are many possible values of the weights that can easily exactly fit a set of examples, but which won't generalize at all.

But the reality is, when we train a network, we're not optimizing over the space of all neural networks.

We're not actually optimizing over all neural networks.

We're actually optimizing over the class of neural networks that in some sense is attainable or considered via SGD, reachable via this one optimization procedure with a given weight initialization.

You remember in our previous example, when we initialized with different weights, the weights didn't actually change all that much, so clearly we're not kind of searching over all possible weight settings.

We're actually thinking about a very limited class of kind of deviations from our initial values, and that is a form of implicit regularization.

Okay, but the kind I'm actually going to talk about here, and to be clear, this happens everything that we do, That's a big exaggeration.

But a whole lot of what we do in designing architectures for deep networks or designing optimizers is about implicitly regularizing the complexity of these functions such that we can generalize better.

What we're going to talk about here, though, in this section is actually going to be explicit regularization, though, which refers to modifications we make to either the network or to training that are explicitly intended to regularize the network, to control the complexity of the network function class.

And this is actually what we're going to talk about here, but I need to emphasize that these are both very common in deep networks.

So the most common form of explicit regularization you will see.

Well, I guess I should take that back, because both the two types I'm going to talk about here are common.

But the most common form of regularization you see regularization you see applied to the weights of a network at least is something called L2 regularization or in deep networks is often also called weight decay and I'll talk about why it's called this in a second. So to

define L2 regularization I'm going to sort of bring us back to our classical machine learning optimization problem. So our problem we had before was

to minimize over our, in this case I'll make our parameters to be explicit, say our parameters are just our weights 1 through L, or my weights I believe, and I

want to minimize the average value of, say, over all my examples of the loss between my prediction and I'm going to write my hypothesis to class as a function of 1 through L,

applied to x_i and y_i . Okay, that's my classical, again, everything machine learning optimization problem. The idea behind regularization, L2 regularization, is that, classically speaking, A key way of assessing the complexity of a hypothesis class is via the size, say just the norm, of these parameters.

And the reason why, and I won't get into this too much because this intuition admittedly does break down a bit when it comes to deep networks.

But one way of thinking about this is imagine your weights were all zero.

Then you basically, by the forms we had before, you're essentially predicting, with no bias terms, say, you're going to predict zero everywhere.

If your weights are small, your functions have to be very smooth.

They can't change very much because you're not applying very large factors to your inputs.

So your function will be very slowly varying over different inputs with small weights.

And so the smaller your weights are, in some very real sense, the smoother your function has to be.

A bit more formally, the size of the weights imposes restrictions on the smoothness or the Lipschitz constant of the function that you're actually using to represent your data.

And what this means is because smoother functions, in some sense are less complex, right?

They can't change as fast, unless that they're less kind of highly varying.

One way to control the complexity of a function is to ensure that the value of the weights themselves are small.

So we want to make the parameters small.

Okay, that's one way of making our, keep controlling the complexity of our function class.

And a very common way to do that is to do this by adding in our loss function, by augmenting our optimization problem to add what's called a regularization term.

The form of this term in the case of L2 regularization is that we're going to add a term plus λ over 2

times the sum from, let's see, a little more space here, the sum from, in this case, I'll use j because I'm indexing here over the, not actually, I'll still use i here because I'm not using two sums, but the sum from i , here's indexing over the layer, so the sum from i equals one to l is indexing over the layers of my network of the norm of my i th layer.

And I guess this is actually a matrix, So technically speaking, I should use what's called the Frobenius norm squared here.

This is just the sum of the squared elements of this vector, of this matrix.

So as a vector, the L2 norm is called the Frobenius norm when you apply it to matrices because you don't want to confuse it with the spectral norm and things like this.

So that's the form of regularization that we often use in or that's sort of most common called L2 regularization. And I should actually be kind of if I'm being fully honest here, I previously referred to this thing here as like the optimization problem of all machine learning.

Minimizing over your data set, the sum of losses, is minimizing over your parameters the sum of losses on your data set.

And really, though, probably more fairly, this thing is the actual true machine learning optimization

problem because very often there is a form of regularization either built in or explicitly considered in machine learning methods.

And so really, regularized minimization losses in the training set is an even more apt description of kind of the single optimization

problem that all machine learning uses, with of course the possibility of other measures of complexity rather than just the norm of the weights. You could have other more generic

regularizers too, but really this is in some sense an even better description of the core optimization problem of all machine learning. Okay, so what happens when we actually use this

update, this form of the update here? So if we define our new optimization problem as this new objective, what we get for our gradient descent update is, well, most of it is like

before. So before our gradient descent update takes our old parameters and subtracts off α , times the gradient of this whole term here.

So it would be the gradient of the loss.

Maybe I'll even write it just for...

I'll write it out with the sum and everything too, but of course this could apply to mini batches and anything else too.

So it would be the gradient with respect to W_i of the sum of our losses, which I'll actually maybe just write as the...

No, I'll write it like that.

Of course you could write it as the sum of the gradients too, doesn't really matter. The loss between our hypothesis w_1 through l , h of x_i , y_i .

So that is the exact same thing as we had before in our gradient descent update.

But then you would also subtract off α times the gradient of this thing.

But the gradient of the norm of the vector squared, this is like one of the standard gradients you probably know,

or I would say what it is, the gradient of the norm of a vector squared is just that vector or that matrix, right?

So what this is, of course, is this is the gradient of this term here, But the gradient of W_i with respect to that whole thing is just equal to λ times W_i .

And that's not very hard to show.

That just is obviously the terms in the sum, because I'm thinking the gradient set for W_i , the terms in that sum other than W_i don't matter, they fall out.

And then the gradient with respect to w_i of the Frobenious norm of w_i , or for a vector the squared norm, is just that vector, or that matrix.

So this is what our gradient descent update becomes.

And if we just refactor terms here, you know, we have a w_i here and a w_i here, So this whole thing can also be written as $1 - \alpha \lambda$ times w_i minus α times our old gradient, right?

The gradient with respect to w_i of the sum from i equals 1 to m .

I guess I should have an average here. I should have a $1/m$ in both places.

doesn't really matter, of our loss, w_1 through w_m , x_i , we're out of space here, y_i .

Okay, barely didn't quite make it. Move things over just a little bit here.

Okay, hopefully you can see that whole thing. Okay, so that's our update.

And the key point that I want to make of this update is that what this update does when you combine terms

is that it takes, instead of just taking our w_i and updating it with our previous w_i , we take, we set w_i equal to some factor times the old w_i .

And this factor, because α is, you know, the step size is considered a small number, and because λ is also what's called

a regularization parameter here, just some factor that trades off between our loss function and the regularization, these are both positive.

And so this thing here subtracts a positive number of $1 - \alpha \lambda$ a positive number.

And so this whole update will essentially shrink W .

Will be a shrunken version of W_i .

In other words, we shrink the weights by this factor, $1 - \alpha \lambda$, where again, λ is our regularization parameter here, before taking each gradient step.

And because of this, because of this sort of shrinkage of the weights here, this is often called weight decay.

But it's important to know what this is really doing is that this is taking the gradient, not just of our loss, but also of this regularization term that involves our weights.

And that's how you derive this L2 regularization updates.

Oftentimes for convention, and I'm not quite sure why it evolved this way, but oftentimes for convention, we implement L2 regularization as a feature of the optimizer, not as a feature of the loss.

I think this is actually a bit wrong.

I personally think that we should define this regularizer as part of our loss function and do things that way.

But that's not the decision that the community made of deep learning.

So what you'll really see frequently is optimizers, SGD or Adam or these things, have a term that is called the weight decay term.

And that is exactly just this parameter λ here, which affects how much you trade off between your weights and your loss function.

And it's often incorporated in the optimizer itself.

But if you're uncertain how to incorporate this in the optimizer, it's really important to know how it's derived this way.

Because if you need to implement weight decay in Adam, what you should not do is try to figure out a different kind of weight decay that is correct for Adam.

What you should just do is take your gradient.

So whenever you have a gradient of your normal system, that's this whole thing here, you just replace it with the way

actually implement weight decay is you replace it with the the gradient of the whole thing including the regularization term and you know for whatever momentum terms or anything else you have you would

just use that whole combination as your gradient so that that's how you implement weight decay correctly with momentum or with Adam or these other things and so that's it's it's important

to know how you come up even though the resulting formula is that you just sort of multiply your weights by a constant less than one before the update, it's important to implement that

kind of in terms of this regularized optimization problem when you actually do it so that you know how to implement this properly for other optimizers as well. All right, so L2 regularization is kind

of the I would argue kind of most common form of weight regularization people use in deep networks, But there is a big caveat here, which is that even though it is very common,

it's unclear how much it really makes sense, right?

Because in practice, it's unclear how much the norm of the weights really affects the complexity of the resulting underlying function.

I mean, I gave this example where if the weights were all zero, yes, the function is not very complex.

But in terms of the variances you get when it comes to real networks, networks, it's much less clear.

So remember this example I gave before of networks initialized with three different weights that all got... so this one, I think I didn't actually include it here,

but this one was a variance of the initialization weights of $2.3/n$, I think. This one was $2/n$. This blue one here was $1.7/n$.

Okay, so we had the different initializations, and these are very different weights, right, when it comes down, maybe not very different, but they are somewhat different weights.

But these networks all get the same loss, and the same actually generalization loss on MNIST in this case.

And so, is it really that important to shrink weights?

It's sort of unclear, right?

And personally, weight decay is very common.

So when people run gradient descent, you will often see some amount of weight decay.

People have just tuned it over time and found that a small value of $1e-4$, is maybe a good, works slightly better than no weight decay at all.

But I often ignore it entirely when I train deep networks.

I often don't bother with weight decay because parameter magnitude, sort of the absolute magnitude of the parameters, especially when you include normalization layers and this kind of stuff, it's often a very bad proxy for complexity in deep networks.

And so while this is very common and you should know about it, and it is in wide use, you will very, very often

see it used in practice, I often find that I don't tend to use it that often in deep networks.

The other form of regularization that I want to mention here, because it's very common, and I do consider this to be a form of regularization, is something called a strategy called dropout.

So if you can think of L2 regularization as a regularizer, an explicit regularizer apply to the weights of a network, the parameters of a network. Another common strategy is to apply

a kind of regularization to the activations of the network. And the essential idea here, and this also maybe seems odd at first, not unlike other things like, say, BatchNorm we considered,

is to, as we are training our network, the idea of dropout is you randomly set some fraction of of the activations in each layer to zero, okay?

So, formally speaking, what I mean by that is, you know, similarly, let my $\hat{z}_i$ here be kind of a tentative value for the next layer,

but my actual value for the next layer would be equal to the j th element here, so that the element j in my next layer

would be equal to $\hat{z}_i$, it just equals that layer, with probability, probability one minus p , and with probability p , I set it to be equal to zero.

This is with probability p .

And that's the idea of dropout, but it's actually not quite complete yet, Because if I do this and kind of random set some to zero,

I am going to be changing the overall weight of these activations.

I'm going to be, with the same initialization, I will say no longer can, I will no longer have weights

that remain, or variances of the weights that remain steady over throughout the network.

So I have to actually also scale these weights by the probability that I keep them.

And so actually the correct scaling for these things maintain the variance properly is just to divide this by $1 - p$. So if I keep, if I set half to

zero, then I'll divide the rest by half to make them twice as large. If I set 0.1 to zero, I'll divide the rest by 0.9 to make them a little bit larger. And that's the idea of dropout.

Now, again, like batch norm, dropout has different performance at training time at test time than at test time.

So dropout is simply only applied at, not typically, though not always, applied at training.

So you're trying to sort of regularize your network at training time.

But it typically is not applied at test time.

So at test time, you just leave all your activations in there.

Now, this does seem very odd, right?

Because it seems like in doing this, we must be massively changing the function that we're actually approximating here, right?

Aren't we kind of completely destroying the function if we just randomly set some of our features to be equal to 0?

And dropout is often cast as kind of a way making the networks robust to missing activations, is one way it's kind of ramed. I never really liked this characterization.

Why should it be robust to missing activations? After all, you don't have these missing activations at test time. Why should this be thought of as regularization?

I think the best way to think about dropout is actually to think of it as a stochastic approximation. And just like we did SGD to approximate gradient descent over our entire

objective, dropout can be thought as a very similar thing applied to the individual activations of a network. So here's what I mean by this. When we did our sort of SGD approximation,

we use the fact, we kind of relied on the fact, and to be clear, stochastic gradient descent it's itself

kind of, I mentioned, this sort of implicit form of regularization. It adds kind of noise to our gradients, which actually does provide a kind of regularization to our function.

And so what we did when we approximated things with SGD is we said that my kind of overall objective here was kind of equal to some sort of similar loss computed only over a mini-batch, so say all i in the batch or something like this of the loss between $h \times i$ and y_i .

This was the stochastic gradient descent approximation that we used.

And in some sense, this is very similar to what's being done with dropout.

So in dropout, you can think of, or really sort of zooming in a bit more on kind of what's really being computed in a layer.

The way we sort of compute each activation prior to its nonlinearity is we take the corresponding elements in a column of w and multiply it by the activations of this vector here.

And so this notation here means that the j th row here times all aspects of this all...

So this would be the j th row here, right, because we're just transposing it, times the activations of this vector here.

And so that's just sort of the broken down version of this matrix multiplication that we do.

And so one way to think about this, about dropout, is that we are just basically doing a stochastic approximation of this individual activation.

We are approximating this thing as similarly equal to z_i plus 1 equals our same nonlinearity applied to the sum, not over all the different weights,

but just over the j in some subset, say p here, where we're going to multiply w_j , all the elements here, times z_{ij} , and then we need to normalize this to

properly have kind of the correct scale just like we normalized by 1 over B here instead of 1 over m . We would sort of divide by the size of our set here, but of course you know we didn't normalize this one

You know this this one was not written as normal as an average version to the sum and so we need to multiply

on the top here by n , and this factor is exactly kind of the the the average factor you get of of one over one, this thing here would just be, of course, equal to one over one minus p .

So this is the factor that we multiply our weights by.

And so I think this is the right way to think about dropout.

It is, dropout is a stochastic, it basically takes the idea of stochastic approximation into the activations of the network itself, the individual layers of a network, network. And in doing so, provides a similar degree of regularization as stochastic gradient descent provides for the traditional training objective. And that's how I like to think about dropout. Okay. So that actually covers the majority of the techniques that I want to

highlight today. So we covered normalization and regularization both as techniques, as ways of sort of making optimization better or making sure that we don't overfit as much to our to our data and

things like this. But I want to end this with a bit of a more kind of higher level picture here talking about kind of the interaction of all these things we've been talking about over the over the past several lectures.

So we talked about optimization, initialization, normalization, regularization.

And the reality is there are many design choices we can make to ease the optimization of deep networks or to make them perform better, et cetera.

We have the choice of the optimizer, the learning rate of the optimizer, the parameters of the optimizer, the momentum term, which optimizer we use.

We have the choice of weight initialization.

What's our variance we use there?

Maybe we should use a uniform versus normal distribution, all these sorts of things.

Do we add normalization?

Do we add layer norm?

Do we add batch norm?

Do we regularize?

Do we use weight decay or do we use dropout, right?

And these don't even include the other tricks that we might cover in later lectures like residual connections or learning rate schedules or many others that I'm probably forgetting.

And so if when you sort of are first approaching The practice of deep learning.

It is very often, it's very common, rather, to sort of feel like, okay, you know, I don't know what I should choose here, so let me, you know, do a grid search over all possible things here, and that's how I'm going to find the right parameters of my network.

And this is, first of all, this is exactly the wrong thing to do.

Definitely don't do that.

There are way too many choices here to do a grid search, and training one network can take long enough, long enough, let alone training 10 to the 8 networks or something like that to sort of grid, cover your whole list of parameters here.

But you would be forgiven, especially when you see people's recipes for training where they have some random values of these things.

You would be forgiven for thinking that people must have just flailed around randomly on GPUs for many thousands of GPU or millions of GPU hours.

And that's what deep learning is all about.

I'm actually going to push back on that a little bit.

I don't think that's actually quite right.

But to give even a bigger impression about one of these things, I want to actually go through an example case that has been confusing to the field, to be honest.

And that is exactly this case of BatchNorm that I talked about.

So BatchNorm was proposed in this paper you see here called batch normalization accelerating deep network training by reducing internal covariate shift. And the abstract I think actually aptly described exactly what batch norm is doing. It says "training deep networks is

complicated by the fact that the distribution of each layer's input changes during training as the parameters of the previous layers change. This slows down the training by requiring lower

learning rates and careful parameter initialization and makes it notoriously hard to train models with saturating nonlinearities.

We refer to this problem as internal covariate shift and address the problem by normalizing layer inputs.

Our method draws strength from making normalization a part of the model architecture and performing normalization for each training mini-batch."

It also, by the way, they mentioned that it eliminates the need for dropout.

So already they're talking about the interaction in some sense between normalization and regularization and these sorts of things.

And BatchNorm is a great example Because BatchNorm undeniably was a good idea, right?

This has been extremely useful in practice, and very oftentimes there are networks that just will not train if you do not use BatchNorm.

And when you put BatchNorm in there, all of a sudden they just start training and performance is much, much improved.

But there has been admittedly a lot of discussion and confusion in the community about what BatchNorm is actually doing, why this is a good idea, why this might be a good thing to really do in practice.

And this was sort of made famous by, I mean, amongst many other things.

Ali Rahimi actually gave a kind of a controversial Test of Time Award at the NeurIPS 2017 conference.

And he also used BatchNorm as an example in some sense of what's wrong with deep learning or the fact his analogy with deep learning is seemingly like alchemy in a lot of ways.

And people are just trying to cook up interesting things.

And I actually don't fully agree with this assessment, but it's sort of a worthy thing to mention here as a perspective.

And so part of his talk said the following.

It says, "here's what we know about batch norm as a field.

It works because it reduces internal covariate shift.

Wouldn't you like to know why reducing internal covariate shift speeds up gradient descent?

Wouldn't you like to see a theorem or an experiment?

Wouldn't you like to know?

Wouldn't you like to see evidence that batch norm reduces internal covariate shift?

Wouldn't you like to know what internal covariate shift is?

Wouldn't you like to see a definition of it?" I think that's actually, by the end, he was maybe being a bit tongue in cheek there.

It's a bit unfair.

I mean, literally in the abstract, they say, they define this, right?

We refer to this phenomenon that they just described as internal covariate shift there.

So maybe that last statement was a bit, I think, a bit unwarranted.

But it's true.

Does it speed up optimization?

Does it reduce this internal covariate shift that they have here?

Is there a reason why it does?

It's sort of unclear.

And in fact, there have been many papers arguing that, no, this reduction of internal covariate shift is actually not what batch norm is really doing.

One famous example actually being paper in, I believe, NeurIPS 2018, arguing that instead, rather than the sort of original view

of BatchNorm, the actual benefit of BatchNorm was in the fact that it makes the landscape of optimization smoother. So the relevant part of this paper says that "the popular belief is that

this effectiveness of BatchNorm stems from controlling the change of layers' input distributions during training to reduce so-called internal covariate shift. In this work, we

we demonstrate that such distributional stability of layer inputs has little to do with the success of BatchNorm and instead we uncover a more fundamental impact of BatchNorm on the training process.

It makes the optimization landscape significantly smoother." This is an argument that BatchNorm is really not about putting features on an even level but it's actually about interacting with optimization and speeding up the optimization process.

But then, and this one, I should admit, I'm biased here.

I'm an author on this paper.

So take this all with a grain of salt.

But a few years after this, actually, a student of mine had a paper on the dynamics of optimization of deep networks under gradient descent.

And rather buried in the paper, to be honest, it's in Appendix K1 is where this fact is mentioned.

But after a lot of discussion, and the paper shows that actually the smoothness of optimization in deep networks does not behave like we are

used to when it comes to traditional optimization. And a key sentence here is that "we conclude that there is no evidence that the use of batch normalization improves either the smoothness

or the effective smoothness along the optimization trajectory." So maybe the statement that batch helps smoothness is not the case. So this is now several years, and it's almost like

you almost can't write a paper on BatchNorm anymore because people just sort of roll their eyes and say, oh, another BatchNorm paper, another paper trying to explain BatchNorm,

you know, haven't we seen a lot of these? And this just goes to show that, you know, These things that are very standard, there is still a lot of unclarity about what they are actually doing despite the fact that they are very widely used throughout all deep learning.

As a very last point, I just want to make another point which is that in recent years actually BatchNorm has seen another kind of life as a technique for adapting to shifting distributions.

So I don't want to get too much into this because this is actually a whole other topic in machine learning.

But one sort of very common, one kind of growing realization, I think, amongst deep learning is that it's very important to understand how methods will do, not just when they're

evaluated on the same distribution as they are trained upon, but when they're evaluated on somehow a different distribution.

This is a setting called distribution shift.

And what people have found is that if you, this is a paper from a paper called TENT.

It's actually a different approach, but they use batch norm as a baseline in this paper here.

So I'm going to use it.

There are other papers just showing that it actually works as a baseline.

And so the point I want to make here is that this plot here This plot here shows a bunch of performances of different classifiers, such as showing their error, when you give corrupt data in different ways.

This shows the error on the original data, and then you can corrupt the data with Gaussian noise or different motion blur, stuff like this, or zooming, and stuff like this.

When you corrupt the images in some sense, the performance of the classifiers drops a lot.

The original classifier, before it got a little less than 25% accuracy [error], I think this is on ImageNet.

And so when you corrupt it, it goes down to 60% error.

So before it was 25% error, now it's 60% error.

What people have found is that if you just run batch norm at test time, so the thing we were trying to avoid doing by computing those running means,

but if you ignore that and just embrace the kind of chaos of having batch dependency at test time, this substantially improves the performance,

not to the level of the original data set, but it substantially improves performance on distribution shift of your classifiers.

And so batch norm, all I will say here, this is probably, that's enough detail on this for now.

But what I will say here is that batch norm is getting this second life as a method for improving the performance of classifiers under distribution shifts.

So this technique developed to help optimization or avoid dropout, which then got viewed instead as a technique for improving optimization, but maybe not,

is now also getting a second life or a third life or whatever as a technique for helping networks be more robust to distributional shift.

So it's just crazy.

I mean, this is a simple one line formula.

It's crazy how much life this idea has gotten over the course of machine learning, or the course of the last five years of deep learning.

So that's actually all I'm going to say on this for now.

But the last point I want to make on this before we end for today is that this whole discussion, And this was just one example.

I covered a whole set of techniques here, different optimizers, different initializations, different normalization techniques, different regularization techniques.

This discussion, just in the case of batch norm, and there are similar discussions, by the way, for dropout and for weight decay and such,

it may give the impression that these things are just random hacks for which we have no real principled understanding and deep learning in some sense is all about

random hacks, right? You try out different things and sometimes they work and how on earth can you figure out what's actually going to work? And I don't want to give that impression,

actually. I think there's been a lot of really excellent scientific experimentation with all of the above. So even if you can't quite analyze it theoretically, you can often analyze these

things empirically in a very scientific, principled manner, and there's been a lot of excellent work that does this. However, it is also true that we do not have yet a complete picture

of how all the different empirical tricks and techniques that I've introduced there really work and how they interact in the real world.

And so this again might give you the impression that in order to get deep learning systems to work, you have to sort of try out all possible combinations to find the one that works.

But I also want to push back against this, because the good news in many cases is that while these things are all useful tools, and you should have all these different, you should

get some empirical experience with them, you need the empirical experience to understand when they may be useful and when they might come into play here.

The good news is that in many cases, it is actually possible to get similarly good results with very different architectural and methodological choices.

So you could have one architecture that uses dropout, one that doesn't, one that uses normalization and one that uses batch normalization and layer norm.

If you understand how to tune your architecture and take greedy steps along its route, there's a very good chance you will end up with very similar performance as potentially a very different method.

And in some sense, I think that one of the lessons of deep learning over the past several years is that, yes, there are advantages for methodologies here.

But also, it's sort of shocking how similar a lot, in many cases, a lot of very different architectures or methods really work.

So these techniques are ones you should all be familiar with.

We're actually going to sort of stop here when it comes to these sort of different tips and training techniques and different variants.

This is, I think, a sufficient knowledge to proceed now to different architectures and stuff like that.

But you should know these tricks and these tips, but you should also know that you can get good performance with a variety of different methods.

And in fact, next lecture, we're going to start diving into some of these, not just more generic but methodological tricks you can use.

But we're going to instead start talking about kind of standard, more structured architectural choices using the example of convolutional networks in the next lecture. So that's it for these sorts of tricks and we'll see everyone next time.